

Atlas Product Requirements Document (PRD)

Version 0.1 – Foundation

Product Name: Atlas

Prepared By: TechInfinnit Studio

Document Owner: Product Team

Status: Draft

Document Information

Field	Value
Product	Atlas
Version	0.1
Type	Product Requirements Document
Prepared By	TechInfinnit Studio
Target Release	MVP v1.0
Confidentiality	Internal

Table of Contents

Part 1 – Product Overview

- 1. Executive Summary
- 2. Product Vision
- 3. Product Goals
- 4. Product Principles
- 5. Success Metrics
- 6. User Personas
- 7. User Journey

Part 2 – Product Modules

1. **Authentication**
2. **User Profile**
3. **Dashboard**
4. **Learning Paths**
5. **Lessons**
6. **Mission Engine**
7. **Guide Mode**
8. **Simplify Mode**
9. **Hint System**
10. **AI Mentor**
11. **Code Editor**
12. **Compiler Engine**
13. **Hidden Test Engine**
14. **Progress Engine**
15. **XP System**
16. **Badges**
17. **Certificates**
18. **Portfolio**
19. **Analytics**
20. **Leaderboards**
21. **Community**
22. **Notifications**
23. **Settings**
24. **Admin Panel**

Part 3 – Engineering

1. **Architecture**
2. **APIs**
3. **Database**
4. **Security**
5. **Performance**
6. **Accessibility**
7. **DevOps**
8. **AI**

Part 4 – Business

1. **Pricing**
2. **Enterprise**
3. **Hiring Platform**
4. **Marketplace**
5. **Future Roadmap**

1. Executive Summary

Atlas is an AI-powered engineering learning ecosystem designed to help learners become professional software engineers through guided practice, real-world software engineering missions, AI mentorship, and production-inspired development workflows.

Atlas focuses on engineering thinking rather than only algorithmic problem solving.

2. Product Goals

Primary Goals

- Make beginners job-ready.
- Replace fragmented learning.
- Build engineering thinking.
- Create portfolio-worthy projects.
- Improve developer confidence.

Secondary Goals

- Corporate training.
- University partnerships.
- Hiring platform.
- Certification ecosystem.

3. Product Principles

Every feature added to Atlas must satisfy at least one of these principles.

- Learn by Building
- Engineering First
- Think Before Code
- Guide Instead of Solving
- AI as Mentor
- Practical over Theoretical
- Portfolio Driven
- Industry Ready

4. User Roles

Student

Learns, solves missions, earns XP, builds portfolio.

Mentor

Reviews work, creates learning content, provides feedback.

Company

Publishes assessments, recruits talent, reviews portfolios.

Administrator

Manages users, missions, AI prompts, analytics, moderation.

5. User Journey

A learner follows this progression:

Register

↓

Select Goal

↓

Choose Learning Path

↓

Complete Lessons

↓

Solve Logic Missions

↓

Build Features

↓

Debug Applications

↓

Receive AI Feedback



Build Portfolio



Earn Certification



Interview Ready



Employment

6. Module Specifications

Every module in Atlas will follow this specification template.

Module Name

Purpose

Business Goal

User Story

User Flow

Functional Requirements

Non-functional Requirements

Permissions

Dependencies

Acceptance Criteria

Edge Cases

Future Enhancements

Analytics Events

API Requirements

Database Tables

AI Behaviour

UI Components

Risks

Open Questions

Example Module

Authentication

Purpose

Allow users to securely access Atlas.

Functional Requirements

Students can register using:

- Email
- Google
- GitHub
- Apple ID

Students can:

- Login
- Logout
- Reset Password
- Verify Email
- Enable Two-Factor Authentication

Non-functional Requirements

Login response under 2 seconds.

99.9% uptime.

Encrypted credentials.

OAuth compliant.

User Story

As a student,

I want to login securely,

So that I can continue my learning progress.

Acceptance Criteria

✓ User receives verification email.

✓ OAuth login succeeds.

✓ Session restored.

✓ Invalid credentials handled.

✓ Account lock after repeated failures.

Analytics

Track

Registration

Login

Drop-off

Verification

Active Sessions

Future

Passkeys

SSO

Organization Login

7. Mission Engine

The Mission Engine is Atlas's core feature.

Responsibilities

- Deliver engineering missions
- Track progress
- Unlock next stages

- **Manage Simplify Mode**
- **Trigger AI feedback**
- **Run hidden tests**
- **Award XP**

Every mission belongs to one:

- **Track**
- **Module**
- **Topic**
- **Difficulty**
- **Engineering Level**

Mission Metadata

Mission ID

Mission Name

Difficulty

Track

Estimated Time

XP Reward

Prerequisites

Skills Tested

DSA Concepts

Framework Concepts

Industry Scenario

Mission Type

AI Prompt

Guide Steps

Hint Levels

Hidden Tests

Success Conditions

Portfolio Artifact

8. Mission Types

Logic Mission

Feature Mission

Architecture Mission

Bug Fix Mission

Optimization Mission

Refactoring Mission

Security Mission

Accessibility Mission

Testing Mission

Deployment Mission

System Design Mission

Interview Mission

9. Learning Tracks (Phase 1)

Programming Fundamentals

Java

Kotlin

Android

Jetpack Compose

Git

Databases

SQL

REST APIs

Future Tracks

Swift

iOS

Flutter

React

Next.js

Node.js

Spring Boot

Python

Go

Rust

DevOps

Cloud

AI/ML

Cybersecurity

Game Development

10. MVP Scope

Included

- **Authentication**
- **Dashboard**
- **Learning Paths**
- **Lessons**
- **Mission Engine**
- **Simplify Mode**
- **Guide Mode**
- **Hint System**
- **Code Editor**
- **Compiler**
- **Hidden Tests**
- **XP**
- **Portfolio**
- **AI Review**
- **Admin Panel**

Excluded

- Team Collaboration
- Hiring Platform
- Live Competitions
- Marketplace
- Voice Mentor
- Mobile Applications

11. Risks

- Overly ambitious MVP
- High infrastructure cost
- AI hallucinations
- Challenge content quality
- Scaling compiler infrastructure
- Maintaining multiple language runtimes

12. Success Metrics

Technical

- Crash-free sessions
- API latency
- Compiler success rate

Learning

- Lesson completion rate
- Mission completion rate
- Guide Mode usage
- Hint usage
- Knowledge retention

Business

- DAU
- WAU
- MAU
- Premium conversion
- Student retention
- Portfolio completion
- Interview success rate

Appendix A – Product Roadmap

Phase 1

Foundations



Phase 2

AI Learning



Phase 3

Engineering Simulator



Phase 4

Collaboration



Phase 5

Hiring Platform



Phase 6

Atlas Ecosystem

Contents:

1. **Authentication PRD**
2. **Learning Engine PRD**
3. **Mission Engine PRD** ★ (the heart of Atlas)
4. **Compiler & Sandbox PRD**
5. **AI Mentor PRD**
6. **Guide Mode & Simplify Mode PRD**
7. **Gamification PRD**
8. **Portfolio & Hiring PRD**
9. **Admin CMS PRD**

Authentication PRD

Authentication Module PRD

Version: 1.0 (Draft)

Priority: P0 (Critical)

Module ID: AUTH-001

Dependencies: None

1. Overview

Purpose

The Authentication Module provides secure identity verification, account creation, authorization, session management, and access control for all Atlas users.

It serves as the entry point into the Atlas ecosystem and establishes the foundation for personalized learning, progress tracking, and role-based permissions.

2. Business Goal

Authentication should:

- Minimize onboarding friction
- Maximize successful registrations
- Ensure account security
- Support enterprise-ready authentication
- Scale to millions of users
- Maintain compliance with modern security standards

3. Supported Roles

Phase 1

- Student
- Mentor
- Admin

Phase 2

- Company
- College
- Recruiter
- Content Creator

Future

- Organization
- Enterprise Admin
- AI Moderator

4. Authentication Methods

Email + Password

Traditional registration.

Google OAuth

Recommended.

GitHub OAuth

Important because developers already have GitHub.

Apple Sign In

Required for future iOS ecosystem.

Future

- Microsoft
- LinkedIn
- Discord
- Passkeys
- Magic Link
- SSO
- Enterprise Login

5. Registration Flow

Landing Page



Sign Up



Choose Login Method



Verify Email



Complete Profile



Choose Career Goal



Choose Learning Track



Dashboard

6. First-Time User Experience

After login

Never send user directly to dashboard.

Instead

Step 1

Welcome



Step 2

Choose Goal

Example

Android Developer

iOS Developer

Frontend

Backend

AI

Cybersecurity



Step 3

Current Skill Level

Complete Beginner

Beginner

Intermediate

Advanced



Step 4

Daily Learning Time

15 min

30 min

1 hr

2 hr

4 hr



Step 5

Target

Internship

Job

Career Switch

College

Placement



Generate Personalized Roadmap

7. User Stories

Student

As a student

I want to sign in quickly

So I can continue learning.

Mentor

As a mentor

I want secure access

So I can review learners.

Admin

As an admin

I want RBAC

So I can manage the platform securely.

8. Functional Requirements

Registration

Must support

- **Email**
- **Google**
- **GitHub**

- **Apple**

Login

Email

OAuth

Remember Me

Multi-device

Password

Forgot Password

Reset Password

Change Password

Password Strength

Password History

Verification

Email Verification

Resend Verification

Verify Status

Session

Active Sessions

Logout All Devices

Device History

Trusted Devices

Security

Rate Limiting

Brute Force Detection

Account Lock

CAPTCHA

Suspicious Login Alerts

9. Non-functional Requirements

Availability

99.99%

Login

<2 seconds

OAuth

<3 seconds

Password Hashing

Argon2id

Encryption

AES-256

TLS

1.3

10. User Profile Creation

After signup

Collect

Name

Username

Country

Timezone

Preferred Language

Programming Experience

Career Goal

Learning Path

GitHub Username (optional)

LinkedIn (optional)

Portfolio (optional)

Avatar

11. Username Rules

3–25 characters

Unique

Letters

Numbers

Underscore

No spaces

Reserved words blocked

12. Password Policy

Minimum

10 characters

Require

Uppercase

Lowercase

Number

Special Character

Disallow common passwords

Check against breach databases (future)

13. RBAC (Role-Based Access Control)

Student

View learning content

Submit missions

Edit profile

Mentor

Everything Student +

Review submissions

Comment

Create draft content

Admin

Everything +

Manage users

Manage missions

Moderate content

Access analytics

14. Session Management

Display

Current Device

Browser

Location (approximate)

Login Time

IP (masked)

User can

Logout

Logout All

Revoke Device

Rename Trusted Device

15. Security Requirements

JWT

Refresh Tokens

CSRF Protection

XSS Protection

SQL Injection Prevention

Rate Limiting

Password Hashing

Encrypted Cookies

Audit Logs

MFA Ready

16. Database Tables

users

profiles

roles

permissions

user_roles

oauth_accounts

sessions

refresh_tokens

email_verifications

password_resets

login_attempts

audit_logs

trusted_devices

17. APIs

Authentication

POST /auth/register

POST /auth/login

POST /auth/logout

POST /auth/logout-all

POST /auth/refresh

GET /auth/me

Password

POST /auth/forgot-password

POST /auth/reset-password

POST /auth/change-password

Verification

POST /auth/send-verification

POST /auth/verify-email

OAuth

GET /auth/google

GET /auth/github

GET /auth/apple

18. UI Screens

Landing

Sign Up

Login

Forgot Password

Reset Password

Verify Email

Complete Profile

Choose Goal

Choose Track

Choose Skill Level

Success

Session Management

Settings

19. Analytics Events

Registration Started

Registration Completed

OAuth Selected

Login Success

Login Failed

Forgot Password

Password Changed

Verification Success

Drop-off Screen

Average Registration Time

20. Edge Cases

- Email already exists
- OAuth email matches existing account
- Username already taken
- Expired verification link
- Expired reset token
- Multiple active sessions
- Deleted account attempting login
- Suspicious IP login
- Network interruption during signup

21. Error Messages

Avoid technical messages like:

Error 500

Instead:

"We couldn't sign you in right now. Please try again in a moment."

Messages should be actionable and user-friendly.

22. Acceptance Criteria

A user can:

- Create an account using any supported method.
- Verify their email successfully.
- Complete onboarding.
- Reach their personalized dashboard.
- Recover their password.
- Manage active sessions.
- Secure their account.

All critical authentication flows must pass automated testing.

23. Future Enhancements

- Passwordless login
- WebAuthn / Passkeys
- Face ID / Touch ID
- Enterprise SSO
- Organization accounts
- Family accounts
- Student verification
- University email verification
- GitHub contribution sync
- Stack Overflow reputation sync

24. Definition of Done

The Authentication Module is considered complete when:

- All login methods are functional.
- Onboarding creates a personalized learner profile.
- Security requirements meet industry best practices.
- Role-based permissions are enforced.
- Session management is available.
- All acceptance tests pass.
- Monitoring and audit logging are in place.

PM Review

As your Product Manager, I would make one major improvement that many platforms overlook.

Don't stop at authentication—start personalization immediately.

The moment a user signs up, Atlas should begin building a Learning Profile that evolves over time.

Instead of storing only:

- Name
- Email
- Password

Atlas should also maintain:

- Preferred learning style
- Weak concepts
- Strong concepts

- Preferred programming language
- Career target
- Daily study habit
- AI interaction history
- Hint usage patterns
- Confidence score
- Learning velocity
- Knowledge graph
- Engineering maturity score

This profile becomes the foundation for AI recommendations, adaptive missions, personalized roadmaps, and future hiring insights. In my opinion, this Learning Profile is one of Atlas's most valuable long-term assets, and we should start designing it from the very first module rather than treating it as an afterthought.

Adaptive Learning Engine PRD (ALE)

Module ID: LE-001

Priority: P0 (Core Module)

Version: 1.0 Draft

1. Purpose

The Adaptive Learning Engine (ALE) is responsible for delivering personalized learning experiences to every learner on Atlas.

Instead of presenting identical content to every student, ALE continuously evaluates a learner's knowledge, behavior, progress, strengths, weaknesses, and career goals to generate an adaptive learning journey.

ALE acts as the intelligence layer connecting lessons, missions, AI mentor, progress tracking, and recommendations.

2. Product Goals

The engine must:

- Personalize learning
- Reduce learning frustration
- Prevent boredom

- Detect knowledge gaps
- Improve retention
- Increase completion rate
- Help students become job-ready

3. Core Philosophy

Every learner is different.

Atlas should never assume:

- Everyone starts at the same level
- Everyone learns at the same speed
- Everyone needs the same roadmap

The learning experience must evolve continuously.

4. Responsibilities

ALE is responsible for:

- Personalized Roadmaps
- Adaptive Difficulty
- Knowledge Tracking
- Revision Planning
- AI Recommendations
- Learning Analytics
- Daily Missions
- Weekly Goals
- Learning Streaks
- Revision Scheduling
- Confidence Estimation
- Career Path Optimization

5. Inputs

The engine continuously receives data from:

User Profile

- Career Goal
- Preferred Language
- Experience Level
- Daily Learning Time

Mission Engine

- Mission Score
- Completion Time
- Hint Usage
- AI Usage
- Submission Attempts
- Error Patterns

AI Mentor

- Weak Concepts
- Strong Concepts
- Common Mistakes
- Confidence Analysis

Progress Engine

- XP
- Badges
- Level
- Streak
- Daily Activity

Compiler Engine

- Compile Errors
- Runtime Errors
- Test Case Failures

Code Analyzer

- Code Quality
- Complexity
- Maintainability
- Naming
- Architecture

6. Outputs

ALE generates:

- Personalized roadmap
- Recommended lesson
- Recommended mission
- Revision reminder
- Difficulty adjustment
- Weekly goals

- AI mentoring suggestions
- Interview readiness score

7. Learning Graph

Atlas should not treat learning as a list.

Instead it should use a Knowledge Graph.

Example:

Variables

|



Conditions

|



Loops

|



Functions

|



Collections

|



Algorithms

|



Android Lists

|



RecyclerView / LazyColumn



Paging

Each concept depends on previous concepts.

Students cannot unlock advanced topics without sufficient mastery.

8. Learning States

Every concept has one of these states:

Not Started



Learning



Practicing



Needs Revision



Mastered



Expert

The state changes dynamically based on performance.

9. Knowledge Model

Every concept stores:

Concept ID

Mastery %

- Confidence %
- Practice Count
- Revision Count
- Last Reviewed
- Average Score
- Hint Usage
- Failure Count
- Estimated Retention
- AI Notes

10. Adaptive Difficulty

Instead of

Easy

Medium

Hard

Atlas uses:

Very Easy

↓

Easy

↓

Comfortable

↓

Stretch

↓

Challenge

↓

Expert

Difficulty adjusts based on learner performance.

11. Learning Recommendations

Every time a learner opens Atlas, ALE generates:

Today's Focus

Recommended Lesson

Recommended Mission

Concept to Revise

Interview Question

Mini Logic Challenge

Real Feature Challenge

Estimated Study Time

12. Learning Path Generation

When a learner joins:

Input:

Goal = Android Developer

Time = 1 hour/day

Experience = Beginner

Target = Internship

Output:

Personalized roadmap including:

- **Java/Kotlin Basics**
- **Logic Building**
- **DSA Essentials**
- **Android Fundamentals**
- **Jetpack Compose**
- **APIs**
- **Architecture**

- Testing
- Projects
- Mock Interviews

13. Learning Modes

Atlas supports multiple modes.

Guided Mode

Maximum assistance.

Ideal for beginners.

Practice Mode

Hints available.

AI Mentor active.

Challenge Mode

Minimal assistance.

Portfolio eligible.

Interview Mode

No hints.

Timed.

Company-style evaluation.

Revision Mode

Focused on weak concepts.

Sprint Mode

Daily missions.

Time-boxed learning.

14. Knowledge Gap Detection

Atlas detects:

Repeated failures

↓

High hint usage

↓

Long completion time

↓

Compile error patterns

↓

Incorrect architecture

↓

Repeated misconceptions

The engine then recommends prerequisite concepts.

15. Learning Analytics

Dashboard includes:

Learning Velocity

Mastery Graph

Weak Concepts

Strong Concepts

Revision Queue

Study Time

Completion Rate

Engineering Readiness

Interview Readiness

16. Revision Engine

Uses spaced repetition.

Example:

Learn today

↓

Revise tomorrow

↓

Revise after 3 days

↓

7 days

↓

14 days

↓

30 days

↓

90 days

Revision timing adapts based on mastery.

17. Daily Plan Generator

Every day ALE creates:

- Lesson
- Logic Exercise
- Engineering Mission
- Revision
- Reflection

Total duration matches the learner's available time.

18. Weekly Goal Generator

Example:

Complete:

- 5 lessons
- 8 logic missions
- 3 feature missions
- 1 debugging mission
- 1 interview mission

19. AI Collaboration

ALE and AI Mentor work together.

ALE identifies what the learner needs.

AI Mentor decides how to teach it.

This separation keeps the system modular.

20. APIs

Examples:

GET /learning/roadmap

GET /learning/today

GET /learning/recommendations

GET /learning/revision

GET /learning/mastery

POST /learning/update

POST /learning/reflection

21. Database Tables

learning_profiles

knowledge_graph

concept_mastery

daily_plans

weekly_goals

revision_queue

recommendations

learning_sessions

learning_reflections

22. Analytics Events

Lesson Started

Lesson Completed

Mission Recommended

Mission Accepted

Revision Completed

Roadmap Updated

Concept Mastered

Knowledge Gap Detected

Difficulty Increased

Difficulty Reduced

23. Acceptance Criteria

The engine must:

- **Generate a personalized roadmap.**
- **Adapt recommendations after each learning session.**
- **Detect weak concepts automatically.**
- **Create daily learning plans.**
- **Schedule revisions.**

- Update mastery levels.
- Integrate with AI Mentor and Mission Engine.

24. Future Scope

- AI-generated learning paths
- Voice-based tutor
- Emotion-aware learning (optional and privacy-conscious)
- Team learning plans
- University curriculum alignment
- Company onboarding tracks
- Offline adaptive learning
- Multi-language localization

★ Product Manager Recommendation (Very Important)

I would add one feature that I believe could become Atlas's signature innovation.

Engineering DNA

Instead of only tracking "completed lessons," Atlas builds a living profile of how a learner thinks.

Examples of signals:

- Debugging ability
- Architecture decisions
- Problem decomposition
- Optimization habits
- Testing mindset
- Consistency
- Code readability
- Learning speed
- Collaboration behavior
- AI dependency
- Growth over time

This profile should never be used to label or limit learners. Its purpose is to personalize learning, highlight progress, and help learners understand their own strengths and areas for improvement.

Over months and years, this becomes a powerful tool for adaptive learning, portfolio insights, and career development. It also gives Atlas something that no traditional coding

platform has: a rich understanding of how a developer grows, not just whether they solved a problem.

Engineering Mission Engine (EME)

Module ID: EME-001

Priority: P0 — CORE PLATFORM

Owner: Product Team

Dependencies

- Adaptive Learning Engine
- Compiler Engine
- AI Mentor
- Progress Engine
- Portfolio Engine

1. Purpose

The Engineering Mission Engine (EME) is the heart of Atlas.

Its responsibility is to convert educational content into interactive engineering experiences that simulate real software development.

Unlike coding challenge platforms that evaluate isolated algorithms, EME presents learners with authentic engineering missions where they must analyze requirements, design solutions, implement features, debug issues, optimize systems, and justify technical decisions.

Every learning activity in Atlas is represented as a mission.

2. Vision

Every learner should feel like they are working inside a real software company.

Instead of solving disconnected questions, learners complete engineering tasks that contribute to products, teams, portfolios, and long-term skill development.

3. Mission Philosophy

Every mission must answer four questions:

1. Why does this problem exist?

2. **Where would this happen in a real product?**
3. **Which engineering concepts are being practiced?**
4. **How will this improve the learner?**

If a mission cannot answer these questions, it should not exist.

4. Mission Lifecycle

Discover Mission



Understand Context



Read Story



Analyze Requirements



Choose Difficulty



Plan Solution



Code



Compile



Run Tests



AI Review



Reflection



Portfolio Update

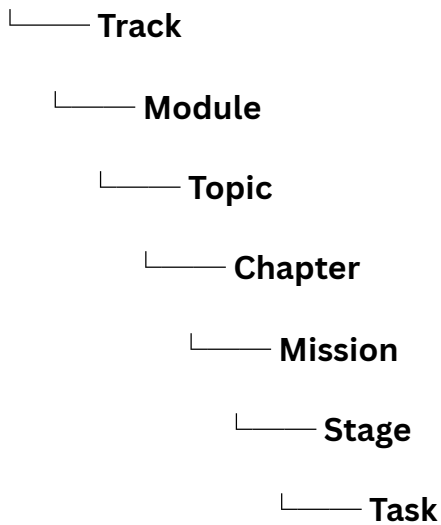


XP & Mastery Update

5. Mission Hierarchy

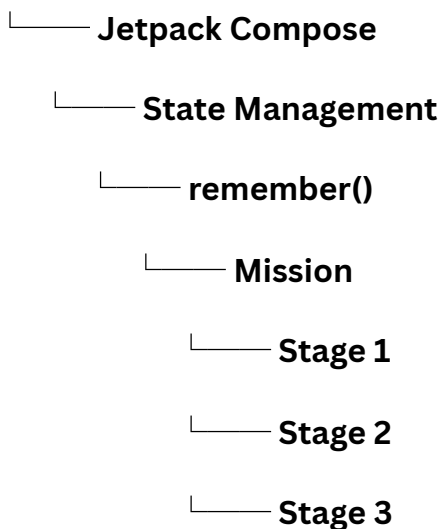
Atlas uses a hierarchical structure.

Career Path



Example:

Android



6. Mission Types

Logic Mission

Purpose:

Build computational thinking.

Example

Sort Instagram followers.

Feature Mission

Purpose:

Implement a real product feature.

Example

Create WhatsApp Status.

Bug Hunt Mission

Purpose

Debug an existing application.

Students receive a broken codebase.

Refactoring Mission

Purpose

Improve code quality.

Performance Mission

Purpose

Reduce memory or CPU usage.

Security Mission

Purpose

Fix vulnerabilities.

Architecture Mission

Purpose

Improve project architecture.

Testing Mission

Purpose

Write unit/UI/integration tests.

System Design Mission

Purpose

Design scalable systems.

DevOps Mission

Purpose

Deploy applications.

Team Mission

Purpose

Collaborate with multiple learners.

Interview Mission

Purpose

Company-style assessments.

7. Mission Template

Every mission must contain:

Mission ID

Title

Difficulty

Estimated Time

Story

Background

Learning Objective

Skills Required

Prerequisites

Constraints

Requirements

Deliverables

Success Criteria

Guide Mode

Simplify Mode

Hint Levels

Hidden Tests

AI Evaluation Rules

Portfolio Artifact

Reflection Questions

XP Rewards

Badges

Certificate Contribution

8. Story Driven Learning

Every mission begins with context.

Bad Example:

Implement Binary Search.

Atlas Example:

"You've joined the Instagram Search team.

Users complain that searching followers becomes slow when they follow more than 50,000 people.

Your task is to improve search performance while keeping memory usage low."

The learner naturally discovers why efficient algorithms matter.

9. Simplify Mode

One of Atlas's signature features.

A complex mission can be decomposed into progressive stages.

Example:

Mission:

Instagram Followers Search

↓

Stage 1

Create User Model.

↓

Stage 2

Load List.

↓

Stage 3

Display Followers.

↓

Stage 4

Search.

↓

Stage 5

Sorting.

↓

Stage 6

Optimization.

↓

Stage 7

Caching.

↓

Final Review.

Learners can exit Simplify Mode once they feel confident.

10. Guide Mode

Guide Mode behaves like a senior engineer.

It never starts with code.

Instead it asks questions such as:

- What inputs does this feature need?
- Which data structure fits best?
- What happens when there are one million users?
- Have you considered edge cases?
- Can this be solved in one pass?

Only after several interactions does it reveal pseudocode or partial implementations.

11. Hint Ladder

Every mission exposes six hint levels.

Level 1 — Clarify the requirement.

Level 2 — Explain the relevant concept.

Level 3 — Suggest an approach.

Level 4 — Provide pseudocode.

Level 5 — Reveal partial implementation.

Level 6 — Reveal complete solution.

Higher-level hints reduce XP but do not prevent progression.

12. Hidden Evaluation

A mission is never judged only by passing tests.

Evaluation includes:

- Functional correctness
- Time complexity
- Memory efficiency
- Readability
- Maintainability
- Architecture
- Naming
- Testing
- Accessibility (where relevant)
- Security (where relevant)

13. Engineering Review

After completion, AI generates a professional engineering review.

Example sections:

- Strengths
- Improvements
- Alternative approaches
- Scalability concerns
- Best practices
- Production considerations

14. Mission Rewards

Every mission contributes to:

- XP
- Coins
- Mastery
- Portfolio
- Streak
- Badges
- Certificates
- Engineering Score

15. Failure Philosophy

Failure is treated as progress.

Atlas does not display:

 Wrong Answer

Instead:

"Your implementation works for small datasets but struggles with larger inputs. Let's investigate why."

The platform encourages iteration rather than discouragement.

16. Portfolio Integration

Every completed mission can generate a portfolio artifact including:

- Mission title
- Skills demonstrated
- Technologies used
- Reflection summary
- Code snapshot
- Engineering review

Learners choose which artifacts become public.

17. APIs

Example endpoints:

GET /missions

GET /missions/{id}

POST /missions/{id}/start

POST /missions/{id}/submit

POST /missions/{id}/hint

POST /missions/{id}/guide

POST /missions/{id}/simplify

GET /missions/{id}/review

GET /missions/recommended

18. Database Entities

missions

mission_stages

mission_tasks

mission_dependencies

mission_hints

mission_guides

mission_templates

mission_rewards

mission_attempts

mission_reviews

mission_reflections

mission_portfolio_artifacts

19. Analytics

Track:

- **Mission starts**
- **Completion rate**
- **Time to complete**
- **Hint usage**
- **Guide Mode usage**
- **Simplify Mode usage**
- **Average attempts**
- **AI review requests**
- **Drop-off points**
- **Mission satisfaction**

20. Acceptance Criteria

The Engineering Mission Engine is complete when it can:

- **Deliver missions based on learner profile.**
- **Support multiple mission types.**
- **Handle staged missions.**
- **Integrate with Guide Mode and Simplify Mode.**
- **Trigger compiler execution.**
- **Receive AI evaluations.**
- **Award progression.**
- **Update portfolio automatically.**
- **Record analytics.**

- Scale to multiple engineering tracks.

★ PM Review — The Feature That Can Make Atlas Unmatched

After everything we've designed, I believe Atlas still needs one defining innovation that competitors cannot easily copy.

Engineering Story Universe (ESU)

Instead of presenting isolated missions, every learner joins a fictional but realistic software company.

Example:

TechInfinit Labs

Learners start as Junior Software Engineers.

Their career progresses through roles such as:

- Intern
- Junior Engineer
- Software Engineer
- Senior Engineer
- Tech Lead
- Staff Engineer
- Principal Engineer
- Engineering Manager

Projects evolve over time:

- Build a messaging app
- Add stories
- Scale to 10 million users
- Handle outages
- Optimize performance
- Introduce monetization
- Expand internationally

Months later, learners are still working on the same evolving products, just as engineers do in real companies.

This creates continuity. Instead of solving 500 unrelated problems, learners experience the growth of software over time—new features, legacy code, technical debt, production bugs, migrations, and scaling challenges.

Why this matters

Today:

- LeetCode teaches algorithms.
- HackerRank teaches coding assessments.
- Frontend Mentor teaches UI implementation.

Atlas could teach what it's actually like to be a software engineer.

In my opinion, the Engineering Story Universe should become Atlas's defining differentiator. If we execute it well, users won't just complete challenges—they'll feel like they're building a career inside the platform long before they land their first real engineering job.

Atlas Secure Execution Platform (SEP)

Module ID: SEP-001

Version: v0.1.0

Priority: P0 – Core Infrastructure

Owner: Platform Engineering Team

Dependencies

- Engineering Mission Engine (EME)
- Adaptive Learning Engine (ALE)
- AI Mentor
- Progress Engine
- Authentication

1. Purpose

The Secure Execution Platform (SEP) is responsible for securely compiling, executing, validating, benchmarking, and evaluating all learner code submitted to Atlas.

SEP provides isolated execution environments that protect the Atlas infrastructure while enabling realistic software engineering practice.

Every code submission passes through SEP before being evaluated or scored.

2. Product Vision

Students should experience professional software development workflows without compromising platform security.

SEP should make code execution feel instant, reliable, secure, and scalable.

3. Core Responsibilities

SEP is responsible for:

- Code compilation
- Secure execution
- Runtime isolation
- Test execution
- Performance benchmarking
- Memory measurement
- Timeout protection
- Resource isolation
- Output collection
- Result reporting
- Artifact cleanup

4. High-Level Architecture

Student

↓

Engineering Mission Engine

↓

Submission Service

↓

Queue

↓

Sandbox Scheduler

↓

Secure Sandbox



Compiler



Test Runner



Performance Analyzer



Result Generator



AI Review



Progress Engine

5. Supported Languages (Phase 1)

Programming

- Java
- Kotlin
- Python
- JavaScript
- TypeScript
- C++
- Go

Mobile

- Kotlin (Android)
- Swift (limited)
- Dart (future)

Backend

- Spring Boot snippets
- Node.js snippets

Future

- Rust

- C#
- PHP
- Ruby
- Scala

6. Execution Flow

Step 1

Receive submission.

↓

Step 2

Validate request.

↓

Step 3

Store source code.

↓

Step 4

Create isolated sandbox.

↓

Step 5

Copy runtime image.

↓

Step 6

Compile.

↓

Step 7

Run hidden tests.

↓

Step 8

Collect metrics.



Step 9

Destroy sandbox.



Step 10

Return report.

7. Sandbox Principles

Every execution must be:

- Isolated
- Disposable
- Stateless
- Ephemeral
- Resource-limited
- Observable

No sandbox should survive after execution.

8. Sandbox Technologies

Preferred architecture:

Container Runtime



Firecracker MicroVM



Docker Image



Language Runtime



Student Code

Alternative runtimes may be evaluated as the platform evolves, but the key requirement is strong isolation with efficient startup times.

9. Resource Limits

Every execution has configurable limits.

Example defaults:

CPU

1 vCPU

Memory

512 MB

Execution Time

10 seconds

Disk

100 MB

Network

Disabled by default

Processes

Limited

Threads

Limited

Environment variables

Restricted

These values vary by mission type.

10. Security Model

Sandbox must prevent:

- File system escape
- Container breakout
- Infinite loops
- Cryptocurrency mining
- Network abuse
- Fork bombs
- Unauthorized process creation
- Environment inspection
- Host access

11. Mission Modes

Algorithm Mode

Compile single file.

Multi-file Mode

Compile project.

Android Component Mode

Compile selected classes.

Backend API Mode

Run service tests.

Full Project Mode

Future.

12. Android Strategy

Atlas should not launch Android Studio for every submission.

Instead

Provide

Prebuilt Project

↓

Student edits

Only

- ViewModel
- Repository
- UseCase
- Composable
- Utilities

SEP injects these into a reference project for compilation and testing.

This dramatically reduces execution time and infrastructure cost.

13. Swift Strategy

Initially support isolated Swift modules and logic.

Full iOS application builds can be introduced later with specialized infrastructure.

14. Test Runner

Runs:

Public Tests

↓

Hidden Tests

↓

Edge Cases

↓

Stress Tests

↓

Performance Tests

↓

Memory Tests

↓

Architecture Rules (where applicable)

15. Performance Metrics

Capture:

Compilation Time

Execution Time

Memory Usage

CPU Usage

I/O Usage

Thread Count

Garbage Collection (language-specific)

16. Output Report

Student receives:

Compilation Result

Runtime Result

Passed Tests

Failed Tests

Performance Summary

Memory Summary

Warnings

AI Suggestions

17. Compiler Errors

Never display raw stack traces by default.

Instead

Explain:

- **What happened**
- **Why it happened**
- **Suggested debugging direction**

Raw logs can be available in an expandable section for advanced users.

18. Queue System

All submissions enter a queue.

Priority

Admin

↓

Live Competition

↓

Premium

↓

Normal

↓

Practice

Supports retries and horizontal scaling.

19. Database Tables

submissions

sandbox_jobs

sandbox_images

compile_results

runtime_results

performance_metrics

execution_logs

hidden_test_results

20. APIs

POST /submission

GET /submission/{id}

GET /submission/{id}/status

GET /submission/{id}/result

POST /sandbox/create

DELETE /sandbox/{id}

GET /sandbox/health

21. Analytics

Track:

Compilation success rate

Average execution time

Average queue time

Runtime failures

Language popularity

Resource usage

Submission frequency

Peak concurrency

22. Non-functional Requirements

Availability

99.95%

Sandbox startup

< 3 seconds

Result delivery

< 5 seconds for standard missions

Horizontal scaling supported

Automatic retries for transient failures

23. Monitoring

Monitor:

CPU

Memory

Disk

Queue length

Container failures

Sandbox failures

Compilation failures

Execution failures

Security events

24. Disaster Recovery

- **Multi-region backups for metadata**
- **Automatic worker replacement**
- **Queue persistence**
- **Failed job recovery**
- **Graceful degradation**

25. Acceptance Criteria

SEP is complete when it can:

- **Securely execute learner code.**
- **Isolate every execution.**

- Support all Phase 1 languages.
- Return detailed execution reports.
- Integrate with EME and AI Mentor.
- Enforce configurable resource limits.
- Scale horizontally.
- Maintain audit logs.

26. Future Scope

- GPU-enabled execution for AI tracks
- Browser-based IDE sessions
- Collaborative execution
- Live pair programming sandboxes
- Custom Docker environments
- Enterprise private runners
- Offline execution packs

Senior Product Manager Review

This module is already strong, but I would make one strategic change that could become one of Atlas's biggest competitive advantages.

Move from "Compile Code" to "Engineering Validation"

Instead of simply checking whether code compiles and passes tests, SEP should support Validation Pipelines.

A mission author could define a pipeline such as:

1. **Compilation**
2. **Unit Tests**
3. **Static Analysis (linting, style, complexity)**
4. **Security Checks (common vulnerabilities where applicable)**
5. **Performance Benchmarks**
6. **Architecture Rules (e.g., MVVM boundaries)**
7. **Accessibility Checks (for UI missions)**
8. **AI Engineering Review**
9. **Portfolio Readiness Score**

Different mission types would use different pipelines.

For example:

- **Android Feature Mission → Compile → UI Tests → Architecture → AI Review**
- **Backend API Mission → Compile → Integration Tests → Performance → Security**
- **Frontend Mission → Build → Visual Regression → Accessibility → AI Review**

This makes Atlas far more flexible than traditional online judges. Instead of a single "compiler," it becomes a configurable Engineering Validation Platform capable of evaluating real software engineering work across Android, iOS, web, backend, DevOps, and future tracks. I believe this architectural decision will make Atlas much easier to extend as new learning paths are added.

Atlas Senior Engineer (ASE)

Module ID: ASE-001

Version: v0.1.0

Priority: Future (Phase 4)

Owner: AI Platform Team

Dependencies

- Engineering Mission Engine
- Adaptive Learning Engine
- Secure Execution Platform
- Portfolio Engine
- Knowledge Graph
- Learning Profile

1. Purpose

Atlas Senior Engineer (ASE) is an AI-powered engineering mentor designed to simulate the guidance of an experienced software engineer.

ASE does not solve problems for learners.

Instead, it helps learners:

- Think critically
- Debug independently
- Improve engineering judgment
- Understand trade-offs
- Build confidence
- Develop long-term software engineering skills

ASE is an educational coach rather than an answer generator.

2. Product Vision

Every learner should feel as though they have an experienced senior engineer available whenever they need guidance.

ASE should encourage curiosity, reasoning, experimentation, and continuous improvement.

3. Core Philosophy

ASE follows one fundamental principle:

Never remove productive struggle.

Learning happens when learners think, experiment, fail, and iterate.

ASE should reduce frustration while preserving the opportunity to develop independent problem-solving skills.

4. Responsibilities

ASE is responsible for:

- Personalized mentoring
- Debugging assistance
- Architecture discussions
- Code reviews
- Engineering explanations
- Career guidance
- Interview preparation
- Learning recommendations
- Reflection facilitation

5. Modes

Mentor Mode

General engineering guidance.

Guide Mode

Mission-specific coaching.

Debug Mode

Analyze compile/runtime errors.

Reviewer Mode

Professional code review.

Interview Mode

Conduct mock technical interviews.

Architect Mode

Discuss design decisions.

Reflection Mode

Help learners summarize lessons.

Career Mode

Recommend skills and roadmap changes.

6. Communication Principles

ASE should:

- Ask questions before giving answers.
- Encourage exploration.
- Explain reasoning.
- Use real-world analogies.
- Recommend documentation.
- Adapt to learner experience.

ASE should avoid immediately revealing implementations.

7. Adaptive Mentoring

ASE receives learner context including:

- Current mission
- Mastery levels
- Learning history
- Preferred language
- Weak concepts
- Hint usage
- Recent mistakes

Responses should adapt accordingly.

8. Mentoring Ladder

ASE should progressively assist.

Level 1

Clarify the problem.

↓

Level 2

Ask guiding questions.

↓

Level 3

Recommend concepts.

↓

Level 4

Suggest debugging strategy.

↓

Level 5

Provide pseudocode.

↓

Level 6

Reveal partial implementation.

↓

Level 7

Reveal complete solution (only after explicit learner confirmation and appropriate mission policy).

Mission settings determine how far ASE may progress.

9. Debugging Assistance

Instead of saying:

"Your code is wrong."

ASE should explain:

- **What failed**
- **Where it failed**
- **Why it failed**
- **Possible causes**
- **Recommended debugging sequence**

10. Code Review

ASE reviews:

- **Readability**
- **Naming**
- **Architecture**
- **Complexity**
- **Performance**
- **Maintainability**
- **Testability**
- **Security**
- **Accessibility**

Feedback should prioritize the most impactful improvements first.

11. Architecture Discussion

Learners can ask:

Why MVVM?

Why Clean Architecture?

Why Repository Pattern?

Why DI?

ASE discusses trade-offs rather than promoting one "correct" answer.

12. Learning Memory

ASE maintains a learning memory containing:

Strong concepts

Weak concepts

Frequently repeated mistakes

Learning preferences

Preferred explanations

Recent goals

Confidence trends

This memory personalizes future guidance.

13. AI Boundaries

ASE should not:

- **Complete graded assessments automatically.**
- **Encourage academic dishonesty.**
- **Generate production credentials or secrets.**
- **Circumvent mission restrictions.**
- **Reveal hidden test cases.**

14. AI Safety

Responses must:

- **Encourage understanding**
- **Respect privacy**
- **Avoid overconfidence**
- **Clearly distinguish facts from assumptions**
- **Handle uncertainty transparently**

15. APIs

Example endpoints:

POST /mentor/chat

POST /mentor/review

POST /mentor/debug

POST /mentor/interview

GET /mentor/history

GET /mentor/recommendations

POST /mentor/reflection

16. Data Models

mentor_sessions

mentor_messages

mentor_feedback

mentor_reviews

mentor_recommendations

mentor_memories

mentor_preferences

interview_sessions

17. Analytics

Track:

- Questions asked
- Average session duration
- Most requested concepts
- Debug requests
- Code reviews
- Interview sessions
- Satisfaction
- Improvement after guidance

18. Acceptance Criteria

ASE is complete when it can:

- Mentor without replacing independent thinking.
- Review code professionally.
- Explain engineering concepts.
- Adapt responses to learner history.
- Support multiple mentoring modes.
- Integrate with the Learning Engine and Mission Engine.

19. Future Scope

- Voice mentoring
- Screen-aware guidance
- Whiteboard collaboration
- Live pair programming
- Team mentoring
- Organization knowledge integration
- Mentor personas (Android Staff Engineer, Backend Architect, etc.)

★ Senior Product Manager Recommendation — The Feature That Could Define Atlas

I believe Atlas can go beyond "AI Chat."

Engineering Memory Timeline

Every learner gets a private engineering timeline.

Imagine a learner asks:

"Why is my RecyclerView lagging?"

ASE responds.

Six months later, the learner asks about Compose performance.

ASE remembers:

"Earlier you struggled with RecyclerView diffing and list rendering. The optimization principles are similar here. Let's build on what you learned."

Or:

"You've improved a lot. Six months ago you needed Guide Level 5 for pagination. Today you solved it independently."

This creates continuity in learning.

The AI isn't just answering questions—it is tracking the learner's engineering growth over months and years.

One final architectural recommendation

I would not implement ASE as a single AI agent.

Instead, design it as an AI orchestration system composed of specialized agents:

- Learning Coach – Explains concepts and plans learning.
- Debugging Engineer – Diagnoses compile/runtime issues.
- Code Reviewer – Reviews style, architecture, and maintainability.
- System Designer – Discusses scalability and architecture.
- Interviewer – Conducts mock interviews.
- Career Coach – Recommends skills and career paths.
- Reflection Coach – Helps learners consolidate knowledge.

The user experiences one "Atlas Senior Engineer," but behind the scenes, the platform routes requests to the most appropriate specialist. This architecture is easier to improve over time and allows Atlas to become significantly more capable without forcing one model to handle every task equally well.

Engineering Guidance System (EGS) & Progressive Mission Decomposition (PMD)

Module ID: EGS-001

Version: v0.1.0

Priority: P0 – Core Learning Module

Dependencies

- Engineering Mission Engine (EME)
- Adaptive Learning Engine (ALE)
- Atlas Senior Engineer (ASE)
- Secure Execution Platform (SEP)

1. Purpose

The Engineering Guidance System (EGS) and Progressive Mission Decomposition (PMD) provide structured assistance that helps learners solve engineering problems independently without immediately exposing solutions.

Instead of replacing critical thinking, the system gradually reduces complexity while preserving learning.

The objective is to maximize understanding rather than completion speed.

2. Product Vision

Every learner should experience the feeling of having an experienced senior engineer sitting beside them.

The mentor does not write the code.

The mentor teaches the learner how to arrive at the solution.

3. Core Philosophy

EGS follows one rule:

Never remove productive struggle.

The learner should always remain the primary problem solver.

Guidance should reduce frustration while maintaining intellectual ownership.

4. Module Responsibilities

The system is responsible for:

- Problem decomposition
- Guided reasoning
- Hint progression
- Concept recommendations
- Adaptive guidance
- AI coaching
- Knowledge verification
- Reflection support

5. Two Independent Systems

Engineering Guidance System (EGS)

Responsible for

- Thinking guidance
- Questions
- Hints
- Debugging
- Explanation

Progressive Mission Decomposition (PMD)

Responsible for

Breaking large engineering missions into smaller engineering milestones.

These systems work together but remain independently configurable.

6. Engineering Guidance Levels

Every mission supports progressive guidance.

Level 0

Independent Mode

No help.

Portfolio eligible.

Level 1

Clarification

Restate the requirements.

Explain the objective.

Level 2

Problem Understanding

Ask questions such as

"What inputs are available?"

"What should happen when the list is empty?"

Level 3

Engineering Thinking

Discuss

Data flow

Architecture

Possible approaches

Trade-offs

Level 4

Planning

Help write

Flowchart

Pseudo-code

Task breakdown

Level 5

Partial Guidance

Reveal only the current step.

Never future steps.

Level 6

Partial Code

Reveal only one isolated implementation.

Level 7

Complete Solution

Only available after learner confirmation or when the mission policy explicitly allows it.

7. Progressive Mission Decomposition

Instead of

Build Instagram Followers

Atlas generates

Stage 1

Create User model.

↓

Stage 2

Display followers.



Stage 3

Load data.



Stage 4

Search.



Stage 5

Sorting.



Stage 6

Optimization.



Stage 7

Edge cases.



Stage 8

Performance.



Stage 9

Accessibility.



Stage 10

Production review.

8. Dynamic Stage Generation

PMD may use predefined stages or generate additional sub-stages when learners repeatedly struggle.

Example

Student repeatedly fails

Sorting

↓

Automatically create

Mini Mission

Comparator Practice

↓

Return to original mission.

9. Learning Recovery

If a learner struggles continuously,

EGS triggers

Recovery Mode

Instead of continuing the mission,

Atlas recommends

- prerequisite lessons
- logic exercises
- concept revision
- micro-missions

After recovery,

the learner resumes the original mission.

10. Guidance Rules

EGS should never:

- Copy the final solution into the editor.
- Reveal hidden tests.
- Skip prerequisite reasoning.
- Ignore learner history.

EGS should always:

- Ask before telling.
- Explain before showing.
- Encourage experimentation.
- Adapt to learner mastery.

11. Context Awareness

Guidance should adapt using:

- Current mission
- Previous attempts
- Hint history
- Compile errors
- Weak concepts
- Preferred language
- Learning goals
- Engineering maturity

12. Engineering Question Bank

Instead of fixed hints,

EGS generates engineering questions.

Examples

"What data structure best supports this operation?"

"What happens with one million records?"

"What is the time complexity?"

"What assumptions are you making?"

"What happens if the network disconnects?"

This develops engineering reasoning.

13. Hint Economy

Hints are treated as learning resources.

Each hint level has configurable effects on:

- XP
- Mastery
- Mission score
- Portfolio eligibility

Mission authors define these policies.

14. AI Collaboration

ASE uses EGS rather than bypassing it.

Instead of

"Here's the code."

ASE asks

"What have you tried so far?"

This maintains learning integrity.

15. Engineering Reflection

After every mission,

EGS asks learners to answer questions like:

- What was the hardest part?
- What would you change?
- Which concept became clearer?
- What would you do differently next time?

Reflections contribute to the learner's Engineering Profile.

16. APIs

Example endpoints

POST /guide/start

POST /guide/question

POST /guide/hint

POST /guide/recovery

POST /guide/reflection

GET /guide/progress

POST /simplify/start

GET /simplify/stages

POST /simplify/complete-stage

POST /simplify/exit

17. Database Tables

guidance_sessions

guidance_steps

guidance_questions

guidance_hints

simplify_sessions

mission_stages

reflection_answers

recovery_plans

18. Analytics

Track:

- Guide usage
- Simplify usage
- Hint level distribution
- Recovery triggers
- Reflection completion
- Stage completion rate
- Average guidance depth
- Mission abandonment after guidance

19. Acceptance Criteria

The module is complete when:

- Learners receive adaptive guidance without immediate solutions.
- Missions can be decomposed into configurable stages.
- Recovery paths are generated automatically.
- Hint progression is enforced.
- Reflections are recorded.
- AI Mentor integrates with EGS.
- Analytics capture guidance effectiveness.

20. Future Scope

- Voice guidance
- Whiteboard reasoning
- Live mentor escalation
- Collaborative guidance for teams
- Personalized hint styles
- Visual architecture coaching
- Interactive debugging sessions
- VR/AR engineering walkthroughs

★ Senior Product Manager Recommendation — Atlas's "WOW" Feature

I believe this module can become the defining experience of Atlas if we add one more concept.

Engineering Thinking Canvas (ETC)

Before a learner writes a single line of code, Atlas opens an Engineering Canvas.

Instead of immediately entering the editor, the learner completes lightweight planning prompts.

For example:

Step 1 — Understand

- What is the problem asking?
- What inputs exist?
- What outputs are expected?

Step 2 — Design

- Which data structures will you use?
- Which architecture fits this feature?
- What edge cases can you identify?

Step 3 — Plan

- Write pseudocode.
- Break the work into subtasks.
- Estimate time complexity.

Step 4 — Build

Only after this planning phase does the code editor unlock.

Why this is powerful

Professional engineers rarely jump straight into coding. They:

1. Understand the requirements.
2. Design a solution.
3. Discuss trade-offs.
4. Plan implementation.
5. Then write code.

By embedding this workflow into Atlas, you're not just teaching programming—you are teaching professional engineering habits from day one.

One final recommendation

I would also introduce Mission Policies.

Each mission author should be able to configure:

- Whether Guide Mode is available.

- Maximum guidance level.
- Whether Simplify Mode is allowed.
- Hint penalties.
- AI response boundaries.
- Portfolio eligibility.
- Time limits.
- Collaboration permissions.
- Accessibility requirements.

This makes the system flexible enough to support beginner practice, university assignments, interview simulations, certifications, and company assessments—all using the same underlying engine without changing the platform architecture.

In my opinion, EGS + PMD + Engineering Thinking Canvas together form the educational core of Atlas and represent one of the strongest differentiators from existing learning platforms.

This is another module where I think we should avoid copying Duolingo, LeetCode, or HackerRank.

Most platforms use gamification to make users stay longer.

Atlas should use gamification to make users become better engineers.

That difference should guide every decision.

Before we write...

I don't like the name Gamification System.

It sounds like badges and XP.

This module is much bigger.

It manages:

- Motivation
- Progress
- Career Growth
- Rewards
- Community
- Learning Habits

I propose:

Engineering Progression System (EPS)

This sounds much closer to what Atlas is actually doing.

Engineering Progression System (EPS)

Module ID: EPS-001

Version: v0.1.0

Priority: P1

Dependencies

- Engineering Mission Engine
- Adaptive Learning Engine
- Engineering Profile
- Portfolio Engine
- Atlas Senior Engineer

1. Purpose

The Engineering Progression System (EPS) motivates learners through meaningful progression rather than addictive engagement.

Every reward in Atlas should represent genuine engineering growth.

Learners should never feel rewarded simply for clicking buttons or logging in.

Progress must reflect learning, practice, consistency, and improvement.

2. Product Vision

Gamification should reinforce engineering habits.

Instead of rewarding activity, Atlas rewards:

- Understanding
- Consistency
- Reflection
- Improvement
- Collaboration
- Professional behavior

3. Core Philosophy

Atlas does not reward

✗ Watching videos

✗ Opening the app

✗ Random clicking

Atlas rewards

✓ Solving problems

✓ Debugging

✓ Helping others

✓ Improving code

✓ Building projects

4. Progression Layers

EPS contains multiple progression systems.

Engineering XP

↓

Engineering Level

↓

Engineering Rank

↓

Career Journey

↓

Achievements

↓

Certificates

↓

Portfolio

↓

Community Reputation

5. Engineering XP

XP represents engineering effort.

Examples

Finish lesson

+10 XP

Logic Mission

+40 XP

Feature Mission

+120 XP

Debug Mission

+90 XP

Interview Mission

+150 XP

Architecture Mission

+180 XP

Reflection

+20 XP

Helping another learner

+25 XP

Mission policies determine exact values.

6. XP Modifiers

Positive

No hints used

Fast completion

Excellent review

High test coverage

Performance optimization

Negative

Excessive hints

Repeated plagiarism violations

Abandoning mission after multiple attempts

Using complete solution mode

Negative modifiers should reduce rewards, not punish curiosity.

7. Engineering Levels

Example

Level 1

Explorer

↓

Level 10

Builder

↓

Level 20

Engineer

↓

Level 35

Senior Engineer

↓

Level 50

Tech Lead

↓

Level 70

Architect

↓

Level 100

Principal Engineer

These are motivational titles only and must not imply real-world credentials.

8. Engineering Career Journey

Instead of simply leveling up,

Learners experience a simulated career.

Example

Intern

↓

Junior Engineer

↓

Software Engineer

↓

Senior Engineer

↓

Lead Engineer

↓

Staff Engineer

↓

Principal Engineer

↓

Distinguished Engineer

Every promotion unlocks new mission types.

9. Mastery

XP ≠ Mastery

Learners may earn XP,

but mastery depends on

Mission completion

↓

Knowledge verification

↓

Revision

↓

Engineering review

↓

Interview performance

This prevents grinding without learning.

10. Badges

Categories

Learning

Problem Solving

Architecture

Testing

Performance

Security

Debugging

Open Source

Consistency

Community

Examples

"First Pull Request"

"Debug Detective"

"Compose Master"

"Performance Optimizer"

11. Streak System

Unlike Duolingo,

streaks should encourage meaningful learning.

A streak increases only if a learner completes a qualifying activity such as:

- Lesson
- Logic mission
- Feature mission
- Revision
- Reflection

Simply opening the app should not maintain a streak.

12. Daily Missions

Every day ALE generates:

- One lesson
- One logic challenge
- One engineering mission
- One revision task

Completing all grants a daily completion bonus.

13. Weekly Challenges

Examples

Debug 5 bugs.

↓

Optimize 3 features.

↓

Write 10 unit tests.

↓

Complete 2 architecture missions.

14. Engineering Reputation

Separate from XP.

Reputation measures community contribution.

Examples

Helping learners

Writing explanations

Reviewing code

Publishing articles

Reporting bugs

Receiving positive feedback

15. Coins

Engineering Coins can unlock

- **Cosmetic profile items**
- **Themes**
- **Avatar customization**
- **Workspace customization**
- **Optional AI credits (subject to business rules)**

Coins should not unlock academic advantages.

16. Leaderboards

Different leaderboards

Global

Country

University

Friends

Weekly

Monthly

Track-specific

Beginner

Advanced

To reduce discouragement, default views should compare learners with similar experience levels.

17. Seasonal Events

Examples

Android Month

↓

AI Week

↓

Hacktober

↓

Debug Championship

↓

Architecture Challenge

18. Engineering Journal

Automatically generated.

Contains

Lessons learned

Mission history

Growth timeline

Engineering reflections

Achievements

Weak concepts

Strong concepts

This journal becomes part of the learner's private history.

19. Milestones

Examples

First Mission

10 Missions

100 Missions

1000 Tests Passed

First Portfolio Project

First Interview Simulation

First Certificate

These milestones celebrate progress without dominating the learning experience.

20. Community Rewards

Reward

Mentoring

↓

Writing blogs

↓

Helping beginners

↓

Code reviews

↓

Answering discussions

Quality should matter more than quantity.

21. APIs

GET /progress

GET /xp

GET /level

GET /career

GET /badges

GET /streak

GET /leaderboard

POST /claim-reward

GET /journal

22. Database Tables

engineering_xp

engineering_levels

engineering_ranks

engineering_careers

badges

badge_progress

streaks

leaderboards

daily_missions

weekly_challenges

reputation

coins

engineering_journal

23. Analytics

Track

XP earned

Daily streaks

Challenge completion

Leaderboard participation

Badge unlocks

Reputation growth

Mission completion

Drop-off after rewards

24. Acceptance Criteria

EPS is complete when

- **XP is awarded through configurable policies.**

- Levels update correctly.
- Career progression is available.
- Mastery remains independent of XP.
- Streaks reflect meaningful learning.
- Leaderboards support segmentation.
- Journal updates automatically.

25. Future Scope

- Team progression
- Company leagues
- University competitions
- Seasonal battle passes
- Mentor reputation
- Marketplace rewards
- Community events
- Open-source contribution rewards

★ Senior Product Manager Recommendation — Atlas's Signature Progression Feature

I believe Atlas should avoid making learners chase arbitrary levels.

Instead, introduce an Engineering Resume that evolves automatically.

Every completed mission contributes evidence such as:

- Features built
- Bugs fixed
- Technologies practiced
- Architecture patterns used
- Tests written
- Performance optimizations completed
- Team missions participated in

Over time, the learner sees a living resume like:

Engineering Experience

- Built 42 production-inspired features
- Resolved 187 software defects
- Designed 15 scalable APIs
- Completed 23 architecture missions
- Optimized 11 applications for performance
- Wrote 640 automated tests
- Worked across Android, Backend, and Cloud tracks

This is far more meaningful than saying:

Level 73

It communicates what the learner has actually accomplished.

Final recommendation

I would separate progression into three independent systems:

1. Progress (XP & Levels) – Motivation and engagement.
2. Mastery (Knowledge & Skills) – Educational achievement.
3. Engineering Experience (Resume & Portfolio) – Career evidence.

Keeping these systems independent prevents learners from "gaming" the platform while ensuring that employers, mentors, and the learners themselves see authentic growth. This separation aligns perfectly with Atlas's philosophy of rewarding genuine engineering development rather than simple activity.

Engineering Passport (EP)

Why?

Because it isn't just a showcase.

It is

- Portfolio
- Resume
- Skill Graph
- Engineering Timeline
- Learning History
- Project Showcase
- Hiring Profile
- Trust Score
- Engineering DNA

It's much more than a portfolio.

Atlas Engineering Passport (EP)

Module ID: EP-001

Version: v0.1.0

Priority: P1

Dependencies

- Engineering Mission Engine
- Adaptive Learning Engine
- Engineering Progression System
- Atlas Senior Engineer
- Authentication

1. Purpose

Engineering Passport is the public and private professional identity of every learner inside Atlas.

Unlike traditional portfolios that rely on manually uploaded projects, Engineering Passport automatically documents a learner's engineering journey through verified work completed on Atlas.

The goal is to provide trustworthy evidence of engineering ability rather than self-reported claims.

2. Product Vision

Every learner graduates from Atlas with a professional engineering profile that demonstrates:

- Skills
- Projects
- Engineering decisions
- Growth
- Experience
- Portfolio
- Interview readiness

without manually creating a resume.

3. Core Philosophy

A portfolio should prove

not claim.

Instead of writing

"I know Android."

Atlas should prove

- Built 18 Android features
- Fixed 47 Android bugs
- Completed 12 Compose missions
- Implemented MVVM 23 times
- Passed architecture reviews

Evidence matters more than claims.

4. Portfolio Types

Private

Visible only to learner.

Contains

AI notes

Mistakes

Weak areas

Reflection

Learning analytics

Public

Shareable.

Contains

Projects

Skills

Achievements

Certificates

Engineering Experience

Portfolio artifacts

Recruiter View

Special hiring dashboard.

Includes

Verified skills

Engineering scores

Project evidence

Code samples

Growth timeline

Interview readiness

5. Engineering Passport Structure

Profile



Engineering Summary



Skills



Projects



Mission History



Engineering Timeline



Experience



Achievements



Certificates



GitHub



Resume



Recommendations

6. Engineering Summary

Automatically generated.

Example

Abhinav has completed 312 engineering missions across Android, Kotlin, and Backend tracks.

Demonstrates strong debugging ability, architecture understanding, and consistent improvement over eight months.

The learner can edit or replace this summary.

7. Engineering Experience

Instead of fake experience,

Atlas generates verified engineering experience.

Example

Android Engineering

Completed

147 Feature Missions

38 Bug Fix Missions

29 Architecture Missions

12 Performance Missions

8. Project Showcase

Every major mission contributes to projects.

Each project contains

Title

Description

Role

Technologies

Architecture

Mission count

Difficulty

Engineering Review

Screenshots

GitHub Repository (optional)

Demo

Reflection

9. Engineering Timeline

Shows learner growth.

Example

2027



Joined Atlas



Completed Kotlin Fundamentals



Built Chat Feature



Completed Android Track



Passed Architecture Review



Completed Internship Preparation



Received Android Certificate

10. Skills

Skills are automatically verified.

Categories

Programming

Frameworks

Architecture

Testing

Databases

Cloud

DevOps

System Design

Soft Skills

Each skill displays

Confidence

Mastery

Practice

Verification

Last Used

11. Engineering DNA

One of Atlas's signature features.

Instead of saying

"Android Developer"

Atlas builds an engineering profile.

Example

Strengths

- Debugging
- Clean Architecture
- Performance

Growing Areas

- Security
- Accessibility

Preferred Patterns

- MVVM
- Repository
- DI

Learning Style

- Visual
- Practice-first

This profile remains private unless the learner chooses to share it.

12. Portfolio Artifacts

Every completed mission can create evidence.

Examples

Feature implemented

Architecture diagram

Reflection

Code review

Performance optimization

Testing report

AI feedback summary

Learners choose which artifacts appear publicly.

13. Engineering Resume

Automatically generated.

Sections

Summary

Skills

Engineering Experience

Projects

Achievements

Certificates

GitHub

Education

Contact

Learners can customize before exporting.

14. Public Portfolio

Unique URL.

Example

atlas.dev/abhinav

Includes

Profile

Projects

Timeline

Skills

Achievements

Certificates

GitHub

Resume download

Contact links

15. Recruiter Dashboard

Recruiters can view

Verified skills

Project evidence

Mission history (public)

Engineering growth

Technology stack

Code samples

Portfolio score

Recruiters never see private learning data.

16. APIs

GET /passport

GET /passport/public/{username}

GET /passport/projects

GET /passport/timeline

GET /passport/skills

POST /passport/share

POST /passport/export

GET /passport/resume

17. Database Tables

engineering_passports

passport_projects

passport_artifacts

engineering_skills

engineering_timeline

engineering_experience

resume_exports

public_profiles

recruiter_views

18. Analytics

Track

Profile views

Resume downloads

Portfolio shares

Recruiter visits

Project clicks

GitHub clicks

Export frequency

19. Acceptance Criteria

Engineering Passport is complete when

- Learners receive an automatically generated engineering profile.
- Public and private views are separated.
- Verified skills are displayed.
- Projects update automatically from completed missions.
- Resume generation works.
- Recruiters can view public evidence without accessing private learning data.

20. Future Scope

- AI-generated portfolio videos
- Personalized cover letters
- LinkedIn synchronization
- GitHub contribution analysis
- Company endorsements
- Team project showcases
- Interview recordings
- Open-source contribution tracking

Senior Product Manager Recommendation — The Feature That Could Change Hiring

I think Atlas should go beyond a portfolio and introduce a Verified Engineering Transcript (VET).

Think of it as the equivalent of an academic transcript—but for software engineering.

Instead of listing course grades, it contains verified engineering evidence such as:

Engineering Activities

- Feature implementations completed
- Bugs diagnosed and resolved
- Architecture reviews passed
- Performance optimization missions
- Security challenges completed
- Testing coverage achieved

Skill Verification

Each skill is backed by evidence:

Dependency Injection

- Practiced in 19 missions
- Last demonstrated: 3 days ago
- Average review score: 9.2/10

Jetpack Compose

- Used in 41 missions
- Verified through 6 portfolio projects
- Strong proficiency

Growth Metrics

Rather than a single score, show trends:

- Problem-solving improvement
- Debugging improvement
- Architecture maturity
- Consistency over time
- Learning velocity

This transcript should be downloadable, shareable, and verifiable.

One final recommendation (Very Important)

I would make Engineering Passport the center of the hiring ecosystem, not resumes.

Imagine a recruiter opening a candidate's profile and immediately seeing:

- What they built
- How they think
- Which engineering practices they've demonstrated
- How they've improved over time
- Evidence supporting every claimed skill

At that point, the conversation shifts from "Tell me about your experience" to "Let's discuss the engineering decisions you made."

If Atlas achieves that, it won't just help people learn software engineering—it could redefine how early-career developers demonstrate their abilities to employers. That aligns closely with Atlas's mission of connecting genuine learning with real-world engineering readiness.

Engineering Talent Network (ETN)

Module ID: ETN-001

Version: v0.1.0

Priority: Future (Phase 5)

Dependencies

- **Engineering Passport**
- **Engineering Mission Engine**
- **Engineering Progression System**
- **Adaptive Learning Engine**
- **Authentication**

1. Purpose

Engineering Talent Network (ETN) connects verified engineering talent with employers through skill-based hiring instead of relying solely on resumes, degrees, or self-reported experience.

ETN provides companies with trustworthy evidence of engineering ability while giving learners opportunities to demonstrate their skills in realistic engineering environments.

2. Product Vision

Hiring should be based on

what you can build

not merely

what you claim you know.

Atlas enables companies to hire based on verified engineering evidence.

3. Core Philosophy

Traditional hiring asks

"What have you done?"

Atlas asks

"Show me how you solve engineering problems."

4. User Roles

Student

Employer

Recruiter

Hiring Manager

University

Administrator

Mentor (future)

5. Hiring Journey

Student



Complete Missions



Engineering Passport



Skill Verification



Talent Pool



Company Discovery



Technical Assessment



Interview



Offer



6. Student Profile

Every candidate includes

Profile

Engineering Passport

Projects

Verified Skills

Mission History (public)

Certificates

Resume

GitHub

LinkedIn

Portfolio

Availability

Preferred Location

Preferred Role

Expected Salary (optional)

7. Employer Dashboard

Employers can

Create Organization

Manage Hiring Team

Create Job Posts

Search Candidates

Review Portfolios

Invite Candidates

Launch Assessments

Track Hiring

Analytics

8. Job Posting

Each posting contains

Role

Experience

Location

Employment Type

Salary Range

Required Skills

Preferred Skills

Mission Requirements

Assessment Type

Deadline

Hiring Stages

9. Candidate Discovery

Recruiters search by

Skills

Track

Technology

Mission Completion

Engineering Level

Mastery

Portfolio

Projects

Availability

Location

University (optional)

10. Verified Skills

Every displayed skill links to evidence.

Example

Kotlin

Verified



Practiced

63 Missions



Latest Activity

2 Days Ago



Average Review

9.1/10

11. Company Assessments

Companies can create

Logic Challenge



Feature Mission



Bug Fix Mission



Architecture Review



System Design



Interview



Team Challenge

Assessments are built using the same mission engine as learning.

12. Engineering Assessments

Example

Company

Spotify



Mission

Implement Playlist Search



Candidate submits



Secure Execution Platform



AI Review



Hiring Report



13. Hiring Report

Recruiters receive

Mission Score

Engineering Review

Performance

Testing

Architecture

Code Quality

AI Summary

Portfolio

Reflection (if shared)

Recruiters should not receive hidden learner analytics unrelated to hiring.

14. Candidate Pipeline

Applied

↓

Assessment

↓

Shortlisted

↓

Interview

↓

Offer

↓

Hired



Archived

Recruiters can customize stages.

15. Internship Programs

Companies can

Publish Internship



Invite Universities



Receive Applications



Conduct Assessments



Hire

16. University Portal

Universities can

Track Student Progress

View Aggregate Analytics (with appropriate permissions)

Recommend Students

Host Internal Assessments

Organize Placement Drives

Student privacy controls must always be respected.

17. Recruiter Search

Filters

Track

Technology

Mission Count

Project Count

Verified Skills

Experience Level

Country

Language

Availability

Portfolio Visibility

18. APIs

GET /jobs

POST /jobs

GET /candidates

GET /candidate/{id}

POST /assessment

POST /invite

GET /pipeline

POST /offer

19. Database Tables

companies

company_users

jobs

applications

candidate_profiles

candidate_skills

candidate_projects

assessments

assessment_results

interviews

offers

hiring_pipeline

20. Analytics

Track

Applications

Assessment Completion

Interview Conversion

Offer Rate

Hiring Time

Candidate Quality

Recruiter Activity

Job Performance

21. Acceptance Criteria

ETN is complete when

- Companies can publish jobs.
- Learners can apply.
- Assessments run through Atlas.
- Recruiters receive evidence-based hiring reports.
- Engineering Passport integrates with hiring.
- Candidate pipelines are configurable.

22. Future Scope

- AI-assisted candidate matching
- Enterprise SSO
- Campus hiring events
- Global talent marketplace
- Freelance opportunities
- Team-based hiring challenges
- Internal mobility for enterprise customers
- API integrations with Applicant Tracking Systems (ATS)

Senior Product Manager Recommendation — The Feature That Could Redefine Technical Hiring

Most hiring platforms rely on resumes, interviews, and coding tests.

Atlas can introduce a Verified Engineering Record (VER).

Every engineering activity becomes verifiable evidence.

Examples

Android

- Built Authentication Flow
- Integrated REST APIs
- Implemented MVVM
- Optimized LazyColumn performance

Backend

- Designed REST APIs

- Implemented JWT authentication
- Wrote integration tests

Each record links back to the actual engineering mission and review.

Engineering Trust Score (ETS)

I would not create a single "overall score" that ranks developers.

Instead, provide multi-dimensional trust signals, for example:

Evidence

- Verified Android features: 42
- Verified backend APIs: 18
- Verified architecture missions: 9

Consistency

- Active learning over 14 months
- Weekly engineering practice: 91%

Engineering Practices

- Testing habit: Strong
- Performance optimization: Intermediate
- Accessibility: Developing

Growth

- Architecture review scores improved from 6.2 → 9.0
- Debugging efficiency improved by 34%

This gives recruiters richer, fairer information than a single number and avoids reducing engineers to one metric.

One final recommendation

I would make ETN invitation-only for employers during the first releases.

Rather than becoming another open job board, focus on partnering with a small group of trusted companies that value skill-based hiring.

This has several advantages:

- Higher-quality opportunities for learners.
- Better feedback loops with employers.
- Easier moderation and trust.

- Stronger reputation for Atlas.

Over time, ETN can evolve into a global engineering talent network where learning, skill verification, and hiring are all connected through the same ecosystem. That creates a much stronger long-term differentiator than simply adding a conventional job board.

Atlas Control Center (ACC)

Module ID: ACC-001

Version: v0.1.0

Priority: PO (Internal Platform)

Owner: Internal Platform Team

Dependencies

- All Atlas modules

1. Purpose

Atlas Control Center (ACC) is the centralized internal platform used to operate, manage, monitor, and evolve the Atlas ecosystem.

ACC enables internal teams to create educational content, configure AI behavior, manage users, publish missions, analyze platform performance, moderate communities, manage companies, and monitor system health.

2. Vision

Every operation inside Atlas should be manageable without writing production code.

Content creators, product managers, support teams, and administrators should be able to manage the platform through ACC.

3. Primary Users

Platform Administrator

Product Manager

Content Engineer

Curriculum Designer

AI Engineer

Support Team

Moderator

Marketing Team

Operations Team

Finance Team

Developer

4. Role-Based Permissions

Super Admin

Complete system access.

Product Manager

Roadmaps

Lessons

Missions

Analytics

Experiments

Content Engineer

Mission Builder

Lesson Builder

Review

Publishing

AI Engineer

Prompt Library

Model Configuration

Evaluation Rules

Knowledge Base

Moderator

Reports

Community

Comments

Users

Support

User Accounts

Tickets

Sessions

Verification

Finance

Subscriptions

Invoices

Revenue

Refunds

Developer

Feature Flags

API Health

System Logs

Deployments (view only unless authorized)

5. Dashboard

Displays

Daily Active Users

New Registrations

Mission Completion

Compiler Health

AI Usage

Revenue

Errors

Support Tickets

System Status

Deployments

6. Content Management

Manage

Tracks

Roadmaps

Lessons

Modules

Topics

Quizzes

Learning Resources

Version History

Localization

Scheduling

7. Mission Builder

One of the most important internal tools.

Create

Mission Type

Story

Requirements

Stages

Guide Mode

Hints

AI Prompts

Hidden Tests

Rewards

Dependencies

Portfolio Artifact

Difficulty

Mission Policies

Preview

Publish

Mission Builder should support templates to reduce repetitive work.

8. Lesson Builder

Supports

Rich Text

Markdown

Code Blocks

Images

Videos

Interactive Widgets

Embedded Simulations

Versioning

Preview

9. AI Control Center

Manage

Prompt Templates

Safety Policies

Response Styles

Model Routing

Evaluation Rules

Knowledge Sources

Feature Flags

Versioning

Approval Workflow

10. Curriculum Manager

Create

Tracks

↓

Modules

↓

Topics

↓

Lessons

↓

Missions

↓

Dependencies

↓

Learning Paths



Career Paths

Supports drag-and-drop restructuring.

11. User Management

View

Accounts

Sessions

Devices

Progress

Subscriptions

Violations

Reports

Support Notes

Impersonation (with audit logging and strict authorization)

12. Company Management

Approve Companies

Manage Recruiters

Jobs

Assessments

Hiring Analytics

Subscription Plans

13. Community Management

Manage

Posts

Comments

Reports

Discussions

Mentors

Study Groups

Events

Moderation Queue

14. Analytics

Learning

Mission

Business

Infrastructure

AI

Hiring

Revenue

Retention

Funnels

Custom Dashboards

15. Experiment Center

Support

Feature Flags

A/B Tests

Rollouts

Beta Features

Regional Releases

Kill Switches

16. Compiler Management

Manage

Language Images

Resource Limits

Queues

Execution Policies

Timeouts

Health

Version Updates

17. Infrastructure Dashboard

Monitor

API

CPU

Memory

Redis

Database

Queues

Storage

CDN

Errors

Latency

18. Notification Center

Manage

Emails

Push Notifications

Announcements

Campaigns

Release Notes

Scheduled Messages

Templates

19. Localization

Manage

Languages

Translations

Regional Content

Date Formats

Currency

RTL Support

20. Search

Global search across

Users

Lessons

Missions

Companies

Reports

Analytics

Tickets

21. Audit Logs

Every important action is logged.

Example

Mission published

↓

Prompt modified

↓

User suspended

↓

Role changed

↓

Subscription refunded

Logs should be immutable and searchable.

22. APIs

GET /admin/dashboard

GET /admin/users

GET /admin/missions

POST /admin/missions

PUT /admin/missions/{id}

DELETE /admin/missions/{id}

GET /admin/analytics

GET /admin/logs

POST /admin/feature-flags

GET /admin/system-health

23. Database Tables

admin_users

admin_roles

admin_permissions

feature_flags

audit_logs

content_versions

mission_templates

system_notifications

experiments

support_tickets

24. Analytics

Track

Mission publish frequency

Lesson edits

Prompt changes

Moderator actions

Support resolution time

Feature flag usage

Experiment performance

Admin activity

25. Acceptance Criteria

ACC is complete when:

- Internal teams can manage content without engineering support.
- Mission Builder supports the full mission lifecycle.
- AI configuration is centrally managed.
- User administration respects role-based permissions.
- Audit logs capture privileged actions.
- Analytics and infrastructure monitoring are available.
- Publishing workflows support review and approval.

26. Future Scope

- Multi-tenant deployments
- White-label support
- Plugin ecosystem
- AI-assisted content generation
- AI-assisted translation
- Enterprise administration
- Custom branding
- Workflow automation

★ Senior Product Manager Recommendation — The Feature That Will Save Thousands of Hours

The most valuable internal feature isn't analytics or user management.

It's a Mission Studio.

Mission Studio should be a no-code/low-code authoring environment where curriculum designers can create complete engineering missions without writing backend code.

A mission creator should be able to configure:

- Story and business context
- Learning objectives
- Difficulty
- Required concepts
- Stage decomposition
- Guide Mode behavior
- Hint progression
- Validation pipeline
- Hidden tests
- AI prompts
- Rewards
- Portfolio artifacts
- Acceptance criteria

The system should automatically generate the underlying mission definition used by the Engineering Mission Engine.

One More Recommendation (Critical)

After working through all of these modules, I think Atlas is growing beyond a typical application.

I recommend introducing a Platform SDK and Plugin Architecture from the beginning.

Imagine that in two years you want to add:

- Blockchain Development Track
- Robotics Track
- Unreal Engine Track
- SAP Track
- Company-specific onboarding
- University-specific curriculum

Instead of modifying the core platform, each becomes a plugin.

Every module we've designed (Learning Engine, Mission Engine, AI, Hiring, Portfolio, Progression) would expose stable APIs that plugins can use.

This architecture gives TechInfini Studio two major advantages:

1. Faster expansion into new technologies and industries without rewriting the platform.
2. An ecosystem opportunity, where trusted partners, universities, or companies could eventually build their own learning tracks on top of Atlas.

That architectural decision doesn't affect the MVP much, but it could dramatically improve Atlas's scalability and long-term business potential. It is much easier to design for extensibility now than to retrofit it years later.

